Ministerul Educaţiei şi Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică şi Microelectronică

Departamentul *Ingineria Software*

Report on laboratory work no. 3

Course: Operating Systems

Topic: The Command Line – Shell, Pipes and Text Filters

Done by: st. gr. FAF-241, Lungu Ilie

Checked by: *Relitman Patricia*

Chişinău – 2026

Contents

Purpose of the work **2**

1 PART 1 – MOVING FAST AND FINDING HELP **2**
1.1 Tab completion and history . . . 2
1.2 Built-in or external command . . . 2
1.3 Finding help without a browser . . . 3
1.4 Observe . . . 3

2 PART 2 – WHAT THE SHELL DOES TO YOUR LINE **3**
2.1 Observe . . . 5

3 PART 3 – REDIRECTION, CHAINING AND JOBS **5**
3.1 Observe . . . 6

4 PART 4 – ANSWERING QUESTIONS WITH ONE-LINERS **7**
4.1 Q1. How many requests failed? . . . 7
4.2 Q2. How many different pages were requested, and which? . . . 7
4.3 Q3. How many requests did each page get? . . . 7
4.4 Q4. Which user or users have the most failed requests, and how many? . . . 7
4.5 Q5. How many requests did user3 make, and how many of them failed? . . . 8
4.6 Q6. Print the last 3 failed requests, showing only time and user. . . . 8
4.7 Q7. Which login shells appear in /etc/passwd, and how many accounts use each? . . . 8
4.8 Q8. Why do ls /etc | wc -l and ls -l /etc | wc -l differ by one? . . . 8
4.9 Bonus . . . 9

5 PART 5 – YOUR FIRST SCRIPT **9**
5.1 Observe . . . 10
5.2 Bonus – running the script with cron . . . 10

Conclusions **11**

PURPOSE OF THE WORK

The purpose of this laboratory work is to work fast in `bash`, to predict what the shell does to a command line before the command runs, to redirect the output where it is needed, and to answer questions about data with one line of small filters joined by pipes, which are then turned into a first shell script. The work is done on Linux, "from the outside": the real operating system is used through its command-line interface. Every command marked PREDICT was first predicted and then run; the outputs are reproduced as figures, and the answers to all the *Observe* boxes and to the questions Q1–Q8 are given in the corresponding parts.

1 PART 1 – MOVING FAST AND FINDING HELP

1.1 Tab completion and history

Pressing Tab asks the shell to complete the word under the cursor. For `ls /usr/sh` followed by Tab (twice) the shell completed the path to `/usr/share/`: there was only one directory beginning with `/usr/sh`, so the second Tab had nothing to list (when several names match, the first Tab completes the common prefix and the second one lists all the candidates). For `cd /et` one Tab was enough to complete `/etc/`.

```
$ ls /usr/sh<Tab><Tab>
$ ls /usr/share/
$ cd /et<Tab>
$ cd /etc/
```

Figure 1.1 – Tab completion

`Ctrl+R` starts the reverse search in the command history: after typing a few letters of an earlier command, the shell shows the most recent command that contains them, and Enter runs it again (pressing `Ctrl+R` again moves to the previous match). This avoids retyping long commands.

1.2 Built-in or external command

```
$ type cd ; type ls ; type echo ; which ls
cd is a shell builtin
ls is /usr/bin/ls
echo is a shell builtin
/usr/bin/ls
```

Figure 1.2 – Output of type and which

```
$ echo $PATH
/usr/local/bin:/usr/bin:/bin:/usr/local/games:/usr/games
$ echo $PATH | tr ':' '\n'
/usr/local/bin
/usr/bin
/bin
/usr/local/games
/usr/games
$ echo $PATH | tr ":" "\n" | wc -l
5
```

Figure 1.3 – The directories in PATH

```
$ nosuchprog ; echo "exit code: $?"
bash: nosuchprog: command not found
exit code: 127
```

Figure 1.4 – Exit code of a missing command

1.3 Finding help without a browser

```
$ ls --help | head -5
Usage: ls [OPTION]... [FILE]...
List information about the FILEs (the current directory by default).
Sort entries alphabetically if none of -cftuvSUX nor --sort is specified.

Mandatory arguments to long options are mandatory for short options too.
$ ls --help | grep "sort by time"
  -t                         sort by time, newest first; see --time
$ help cd | head -3
cd: cd [-L|[-P [-e]] [-@]] [dir]
    Change the shell working directory.
```

Figure 1.5 – Help for ls and cd without a browser

Help is available from `man` (the manual page of a command, with `/word` to search and `n` for the next match), from the `-help` option of external commands and from the built-in `help` for shell built-ins, as the figure shows.

1.4 Observe

1. **Built-ins.** `cd` and `echo` are shell built-ins, while `ls` is an external program (`/usr/bin/ls`). `cd` *must* be a built-in: every program runs in a child process that has its own working directory. If `cd` were an external program, it would change the directory only of its own process, which ends immediately, and the shell would stay in the old directory. Only a command executed by the shell itself can change the shell's current directory.

2. **PATH.** On the test machine `PATH` contains 5 directories, and `ls` is found in `/usr/bin`.

3. **Exit code of a missing command.** A command that is not found returns the exit code 127 ("command not found").

4. **Sorting by modification time.** The option `-t` ("sort by time, newest first").

2 PART 2 – WHAT THE SHELL DOES TO YOUR LINE

A test directory with a few files was created:

```
$ touch a.c b.c file1.txt file2.txt notes.txt "alfa beta.txt"
$ ls
a.c
alfa beta.txt
b.c
file1.txt
file2.txt
notes.txt
```

Figure 2.1 – Files created in the play directory

For every line the prediction was written first and then compared with the real output (Table 1).

Table 1: Predictions and real results for the lines of Part 2.

Command	My prediction	Real output	Explanation
`echo *.c`	`a.c b.c`	`a.c b.c`	The shell expands the glob `*.c` to the matching names before `echo` starts.
`echo file?.txt`	`file1.txt file2.txt`	`file1.txt file2.txt`	`?` matches exactly one character.
`echo [a-b]*`	`a.c alfa beta.txt b.c`	`a.c alfa beta.txt b.c`	`[a-b]` matches one character from the range; the names are sorted, and the name with a space stays one word.
`echo *.xyz`	`*.xyz`	`*.xyz`	No file matches, so the shell leaves the pattern unchanged and `echo` prints it literally.
`ls alfa beta.txt`	`error: two missing files`	`ls: cannot access 'alfa': ... / ls: cannot access 'beta.txt': ...`	The space splits the line into two arguments, `alfa` and `beta.txt`.
`ls "alfa beta.txt"`	`alfa beta.txt`	`alfa beta.txt`	The quotes keep the space inside a single argument.
`echo '$HOME' "$HOME" \{}$HOME`	`$HOME /home/... $HOME`	`$HOME /root $HOME`	Single quotes and the backslash keep the dollar sign literal; double quotes allow the variable to be expanded.
`echo {1..5} ; echo $((7 * 6))`	`1 2 3 4 5 / 42`	`1 2 3 4 5 / 42`	Brace expansion produces the sequence; `$((...))` is arithmetic expansion.
`echo "today is $(date +%A), I am $(whoami)"`	`today is <weekday>, I am <user>`	`today is Thursday, I am root`	Command substitution runs `date` and `whoami` and inserts their output.
`touch log{1..3}.txt && ls log*`	`log1.txt log2.txt log3.txt`	`log1.txt log2.txt log3.txt`	The braces create three names for `touch`; `log*` then matches them.

```
$ echo *.c
a.c b.c
$ echo file?.txt
file1.txt file2.txt
$ echo [a-b]*
a.c alfa beta.txt b.c
$ echo *.xyz
*.xyz
```

Figure 2.2 – Globbing results

```
$ ls alfa beta.txt
ls: cannot access 'alfa': No such file or directory
ls: cannot access 'beta.txt': No such file or directory
$ ls "alfa beta.txt"
alfa beta.txt
$ ls alfa\ beta.txt
alfa beta.txt
```

Figure 2.3 – The ls error on a name with a space, and the two fixes

```
$ echo '$HOME' "$HOME" \$HOME
$HOME /root $HOME
```

Figure 2.4 – Single quotes, double quotes and backslash

```
$ echo {1..5} ; echo $((7 * 6))
1 2 3 4 5
42
$ echo "today is $(date +%A), I am $(whoami)"
today is Thursday, I am root
$ touch log{1..3}.txt && ls log*
log1.txt
log2.txt
log3.txt
```

Figure 2.5 – Brace expansion, arithmetic and command substitution

```
$ ls
 a.c      b.c        file2.txt   log2.txt   notes.txt
'alfa beta.txt'   file1.txt   log1.txt    log3.txt
```

Figure 2.6 – Output of ls on a terminal: the name with a space is quoted

2.1 Observe

1. The predictions and the real outputs are in Table 1.
2. **Who produced the names for `echo *.c`?** The *shell*. The pattern is expanded (pathname expansion, "globbing") *before* the command is started; `echo` only receives the already expanded list `a.c b.c` as arguments and prints them, without knowing anything about files.
3. **Why did `ls alfa beta.txt` fail?** The shell splits the line at spaces, so `ls` received two arguments, `alfa` and `beta.txt`, and none of them exists. Two fixes: put the name in quotes, `ls "alfa beta.txt"`, or escape the space with a backslash, `ls alfa\ beta.txt`.
4. **Single and double quotes.** Inside single quotes everything is literal: no variables, no command substitution. Inside double quotes the spaces and the wildcards are protected from the shell, but `$variable`, `$(command)` and the backslash are still processed. The backslash escapes only the next character.

Why does `ls` show `'alfa beta.txt'` in quotes? When the output goes to a terminal, GNU `ls` quotes the names that contain spaces or other special characters, so that the shown name can be copied and pasted directly into a command and still be read by the shell as one word.

3 PART 3 – REDIRECTION, CHAINING AND JOBS

```
$ ls /etc /nope > out.txt 2> err.txt
$ wc -l out.txt ; cat err.txt
141 out.txt
ls: cannot access '/nope': No such file or directory
$ ls /etc /nope > all.txt 2>&1 ; tail -2 all.txt
xdg
xml
```

Figure 3.1 – Standard output and standard error in separate files

```
$ echo first > notes.md ; echo second >> notes.md ; cat notes.md
first
second
$ wc -l < access.log
500
$ wc -l access.log
500 access.log
```

Figure 3.2 – Append, input redirection and the file name printed by wc

```
$ ls /etc /nope > all.txt 2>&1
$ wc -l all.txt; grep -c nope all.txt
142 all.txt
1
$ ls /etc /nope 2>&1 > all2.txt
ls: cannot access '/nope': No such file or directory
$ wc -l all2.txt; grep -c nope all2.txt
141 all2.txt
0
```

Figure 3.3 – Comparing the order of redirections

```
$ ls /nope 2> /dev/null || echo "failed with code $?"
failed with code 2
$ ls /etc > /dev/null && echo "listing ok"
listing ok
$ true ; echo "after ;"
after ;
$ mkdir -p tmp && cd tmp && echo "I am in $(pwd)" ; cd ~/os-lab3
I am in /root/os-lab3/tmp
$ pwd
/root/os-lab3
```

Figure 3.4 – The operators ; && and ||

```
$ sleep 100 &
$ jobs
[1]+  Running                 sleep 100 &
$ kill %1 ; jobs
[1]+  Running                 sleep 100 &
[1]+  Terminated              sleep 100
$ jobs
```

Figure 3.5 – Running a background job and stopping it

3.1 Observe

1. With `ls /etc /nope > out.txt 2> err.txt`, the listing of `/etc` (141 lines) went to `out.txt`, and the error message `cannot access '/nope'` went to `err.txt`. In `ls /etc /nope > all.txt 2>&1` both streams are in `all.txt`; the error line is the *first* line of the file (the error is written immediately, the listing afterwards), which is why `tail -2` shows only the last two names of `/etc`.

2. **`> all.txt 2>&1` versus `2>&1 > all.txt`.** The shell processes the redirections from left to right. In `> all.txt 2>&1` the standard output is first sent to the file, and then the standard error is made a copy of the standard output, i.e. it also goes to the file: the file has 142 lines and contains the error. In `2>&1 > all.txt` the standard error is first made a copy of the standard output *while it still points to the terminal*, and only afterwards the standard output is sent to the file: the error appears on the screen and the file has only the 141 lines of the listing (zero lines with `nope`).

3. `wc -l < access.log`: the shell opens the file and `wc` reads it from the standard input, so it never learns the file name and prints only the number. In `wc -l access.log` the file name is an argument, so `wc` prints it after the number.

4. **Operators.** `;` runs the commands one after another regardless of the result of the previous one. `&&` runs the next command only if the previous one succeeded (exit code 0). `||` runs the next command only if the previous one failed (non-zero exit code); in the figure, `ls /nope` fails with code 2, so the `echo` after `||` runs.

4 PART 4 – ANSWERING QUESTIONS WITH ONE-LINERS

The data set was generated with the block from the assignment; the check printed the required `500 access.log`.

```
$ mkdir -p ~/os-lab3 && cd ~/os-lab3
$ head -3 access.log ; wc -l access.log
2026-10-05 10:01 user1 OK /page1
2026-10-05 10:02 user2 OK /page2
2026-10-05 10:03 user3 OK /page3
500 access.log
```

Every question is answered with one command line; all the lines are saved in `oneliners.sh`, with the question as a comment above each of them.

4.1 Q1. How many requests failed?

```
$ grep -c FAIL access.log
100
```

Figure 4.1 – Number of failed requests

The option `-c` counts the matching lines: 100 requests failed.

4.2 Q2. How many different pages were requested, and which?

```
$ echo "$(cut -d' ' -f5 access.log | sort -u | wc -l) pages:" $(cut -d' ' -f5 access.log |
    sort -u)
4 pages: /page0 /page1 /page2 /page3
```

Figure 4.2 – Different pages

Field 5 is the page; `sort -u` keeps one copy of each. There are 4 pages: `/page0`–`/page3`.

4.3 Q3. How many requests did each page get?

```
$ cut -d' ' -f5 access.log | sort | uniq -c
    125 /page0
    125 /page1
    125 /page2
    125 /page3
```

Figure 4.3 – Requests per page

`uniq -c` counts the equal adjacent lines (hence the `sort`). Each page got 125 requests; $4 \times 125 = 500$, as required.

4.4 Q4. Which user or users have the most failed requests, and how many?

```
$ grep FAIL access.log | cut -d' ' -f3 | sort | uniq -c | sort -rn | awk 'NR==1{m=$1} $1==
    m'
     15 user5
     15 user3
```

Figure 4.4 – Users with the most failures

The failures are counted per user and sorted in descending order; the last filter keeps all the lines that have the maximum count, so a tie is shown correctly. `user3` and `user5` have 15 failures each. Check: the per-user counts are $15 + 15 + 5 \times 14 = 100$, equal to Q1.

4.5 Q5. How many requests did user3 make, and how many of them failed?

```
$ echo "user3: $(grep -cw user3 access.log) requests, $(grep -w user3 access.log | grep -c
    FAIL) failed"
user3: 72 requests, 15 failed
```

Figure 4.5 – Requests and failures of user3

`-w` matches the whole word. `user3` made 72 requests, 15 of them failed.

4.6 Q6. Print the last 3 failed requests, showing only time and user.

```
$ grep FAIL access.log | tail -3 | cut -d' ' -f2,3
10:10 user0
10:15 user5
10:20 user3
```

Figure 4.6 – Last three failed requests

`tail -3` keeps the last three failed lines and `cut -f2,3` the time and the user.

4.7 Q7. Which login shells appear in /etc/passwd, and how many accounts use each?

```
$ cut -d: -f7 /etc/passwd | sort | uniq -c | sort -rn
     19 /usr/sbin/nologin
      3 /bin/bash
      1 /bin/sync
```

Figure 4.7 – Login shells in /etc/passwd

The shell is field 7 of `/etc/passwd`, separated by `:`. The result depends on the machine; on the test machine most accounts use `/usr/sbin/nologin`, i.e. they cannot log in.

4.8 Q8. Why do ls /etc | wc -l and ls -l /etc | wc -l differ by one?

Prediction: the long format prints one extra line.

```
$ ls /etc | wc -l
140
$ ls -l /etc | wc -l
141
$ ls -l /etc | head -1
total 692
```

Figure 4.8 – The extra line of ls -l

The long format `ls -l` prints, before the entries, a header line `total N` (the total number of disk blocks used by the listed files). This line is not a file, but `wc -l` counts it, so the result is larger by exactly one.

4.9 Bonus

With `grep -E`, the lines where the minute ends in 0 (10:00, 10:10, . . . , 10:50) and the request failed are selected:

```
$ grep -E ' 10:[0-5]0 .*FAIL' access.log | wc -l
50
```

There are 50 such requests. They are exactly the requests numbered by the multiples of 10: the failures are the multiples of 5, and the minute is the number modulo 60, which ends in 0 for the multiples of 10.

```
$ bash oneliners.sh
100
4 pages: /page0 /page1 /page2 /page3
    125 /page0
    125 /page1
    125 /page2
    125 /page3
     15 user5
     15 user3
user3: 72 requests, 15 failed
10:10 user0
10:15 user5
10:20 user3
     19 /usr/sbin/nologin
      3 /bin/bash
      1 /bin/sync
140
141
total 692
50
```

Figure 4.9 – Full run of oneliners.sh

The checks ask that the counts of Q3 add up to 500 (4×125) and that the per-user failure counts behind Q4 add up to Q1 ($15 + 15 + 14 \cdot 5 = 100$); both hold.

5 PART 5 – YOUR FIRST SCRIPT

The loop for the `TODO` goes through the users `user0`–`user6`, counts the failures of each with a pipeline from Part 4 and prints them:

```
$ cat report.sh
#!/bin/bash
# usage: ./report.sh access.log
log=$1
if [ ! -f "$log" ]; then
    echo "no such file: $log" >&2
    exit 1
fi
total=$(wc -l < "$log")
fails=$(grep -c FAIL "$log")
echo "$total requests, $fails failed"
for n in 0 1 2 3 4 5 6; do
    echo "user$n: $(grep -w "user$n" "$log" | grep -c FAIL) failures"
done
```

Figure 5.1 – Contents of report.sh

```
$ ./report.sh access.log
bash: ./report.sh: Permission denied
```

Figure 5.2 – Running the script before chmod +x

```
$ chmod +x report.sh
$ ./report.sh access.log
500 requests, 100 failed
user0: 14 failures
user1: 14 failures
user2: 14 failures
user3: 15 failures
user4: 14 failures
user5: 15 failures
user6: 14 failures
```

Figure 5.3 – Running the script on access.log

```
$ ./report.sh nope.log ; echo "exit code: $?"
no such file: nope.log
exit code: 1
$ report.sh access.log
bash: report.sh: command not found
```

Figure 5.4 – Error handling for a missing file, and the call without ./

The totals agree with Part 4: 500 requests, 100 failed, and the per-user failures (14 or 15) add up to 100.

5.1 Observe

1. `#!/bin/bash` (the "shebang") tells the kernel which interpreter must run the file: the script is executed by `/bin/bash`. Before `chmod +x` the file does not have the execute permission, so the attempt ends with `Permission denied`.

2. A name without a path is searched only in the directories of `PATH`, and the current directory is deliberately not there (a program placed in a directory could otherwise replace a standard command). `./report.sh` gives an explicit path, so no search is done; `report.sh` alone gives `command not found`.

3. `>&2` sends the message to the standard error, so that error messages are not mixed with the normal output when it is redirected or piped. `exit 1` ends the script with a non-zero exit code, which tells the caller that it failed; the codes are used by `$?`, `&&` and `||` (the figure shows the exit code 1).

5.2 Bonus – running the script with cron

The following line was added with `crontab -e`; it runs the script every minute with full paths and appends both the output and the errors to `report.log`:

```
* * * * * /root/os-lab3/report.sh /root/os-lab3/access.log >> /root/os-lab3/report.log
    2>&1
```

Figure 5.5 – The crontab line

After about two minutes the log contained two complete runs of the report:

```
$ cat ~/os-lab3/report.log
500 requests, 100 failed
user0: 14 failures
user1: 14 failures
user2: 14 failures
user3: 15 failures
user4: 14 failures
user5: 15 failures
```

```
user6: 14 failures
500 requests, 100 failed
user0: 14 failures
user1: 14 failures
user2: 14 failures
user3: 15 failures
user4: 14 failures
user5: 15 failures
user6: 14 failures
```

Figure 5.6 – The log written by cron

The line was then removed with `crontab -r` (afterwards `crontab -l` prints `no crontab for root`).

CONCLUSIONS

In this laboratory work the command line was used as a programming environment. It was seen that the shell, not the command, performs the expansion of globs, variables, braces and command substitution, and that quoting decides which of them happen; that every process has three standard streams, which are redirected from left to right (`> file 2>&1` is not the same as `2>&1 > file`); and that `;`, `&&` and `||` chain commands according to their exit codes (0 means success, 127 "not found").

Small filters (`grep`, `cut`, `sort`, `uniq`, `wc`, `head`, `tail`) joined by pipes answered eight questions about a log of 500 lines, and the results were verified by consistency checks (the counts add up to 500 and to the number of failures). Finally, the best one-liners were turned into the script `report.sh`, which validates its argument, reports errors on the standard error with a non-zero exit code, and can be run periodically by `cron`.